

- Prerequisites**
- Kiro IDE 0.7+ installed — download from [kro.dev/downloads](#)
 - AWS CLI version 2.0+ configured with `aws configure`
 - Python 3.10 or newer

Build an Agent with Amazon Bedrock AgentCore Power

Introduction

The **Amazon Bedrock AgentCore Power** enables you to build, deploy, and operate production-ready AI agents using AWS's managed agentic platform. This Power provides specialized knowledge for working with AgentCore Runtime, Memory, Gateway, and Observability features.

What You'll Learn

In this hands-on lab, you'll:

- Install and activate the Amazon Bedrock AgentCore Power
- Create and test an AI agent locally using Kiro
- Deploy your agent to Amazon Bedrock AgentCore
- Test the deployed agent via Kiro chat
- Build a chat web UI integrated with your agent

Region Consistency
This lab uses `us-east-1`. Ensure your **AWS CLI** default region and Bedrock model access are both configured for `us-east-1`.

Step 1: Set Up Your Workspace

1.1 Create a Project Folder

Create a new folder for this lab and open it in Kiro IDE:

```
1 mkdir kiro-bedrock-agentcore-power
2 cd kiro-bedrock-agentcore-power
3 kiro
```

Enable the Kiro CLI command
If `kiro` doesn't work, you need to install the CLI alias first:

- Open Kiro IDE
- Open the Command Palette (`cmd+shift+p` on macOS / `ctrl+shift+p` on Windows/Linux)
- Search for "Install 'kiro' command in PATH" and select it

After that, restart your terminal and `kiro` will work from any directory.

1.2 Configure AWS Credentials in Kiro Terminal

If you are running this lab in an AWS Workshop environment on your local machine, you need to configure the `awsParticipateRole` credentials so the Kiro terminal can interact with AWS services.

- Go to your Workshop Studio event page
- Click **Get AWS CLI credentials** in the left sidebar
- Copy the credentials block (Option 1: export environment variables)

Open the Kiro integrated terminal (`ctrl+`` or **Terminal** → **New Terminal**) and configure the credentials to persist across all terminal sessions:

macOS / Linux (bash)

```
1 echo "export AWS_ACCESS_KEY_ID='your-access-key'" >> ~/.bashrc
2 echo "export AWS_SECRET_ACCESS_KEY='your-secret-key'" >> ~/.bashrc
3 echo "export AWS_SESSION_TOKEN='your-session-token'" >> ~/.bashrc
4 echo "export AWS_DEFAULT_REGION='us-east-1'" >> ~/.bashrc
5 source ~/.bashrc
```

macOS (zsh — default shell)

```
1 echo "export AWS_ACCESS_KEY_ID='your-access-key'" >> ~/.zshrc
2 echo "export AWS_SECRET_ACCESS_KEY='your-secret-key'" >> ~/.zshrc
3 echo "export AWS_SESSION_TOKEN='your-session-token'" >> ~/.zshrc
4 echo "export AWS_DEFAULT_REGION='us-east-1'" >> ~/.zshrc
5 source ~/.zshrc
```

Windows (PowerShell)

```
1 [System.Environment]::SetEnvironmentVariable("AWS_ACCESS_KEY_ID", "user")
2 [System.Environment]::SetEnvironmentVariable("AWS_SECRET_ACCESS_KEY", "user")
3 [System.Environment]::SetEnvironmentVariable("AWS_SESSION_TOKEN", "user")
4 [System.Environment]::SetEnvironmentVariable("AWS_DEFAULT_REGION", "us-east-1", "user")
```

Restart Kiro after setting user-level variables on Windows for them to take effect.

Important
Workshop Studio credentials are temporary and expire after the event. Remember to remove them after the workshop (see Step 5).

Set the default region to `us-east-1`:

```
1 aws configure set region us-east-1
```

Verify the role is active:

```
1 aws sts get-caller-identity
```

You should see output containing `awsParticipateRole` in the ARN, confirming the terminal has the correct permissions for AgentCore operations.

Running on your own account?
If you are not using an AWS Workshop environment, ensure your terminal has AWS credentials configured with `AdministratorAccess` or the `AmazonBedrockAgentCoreFullAccess` managed policy. Run `aws configure` to set up your credentials.

1.3 Install the AgentCore Power

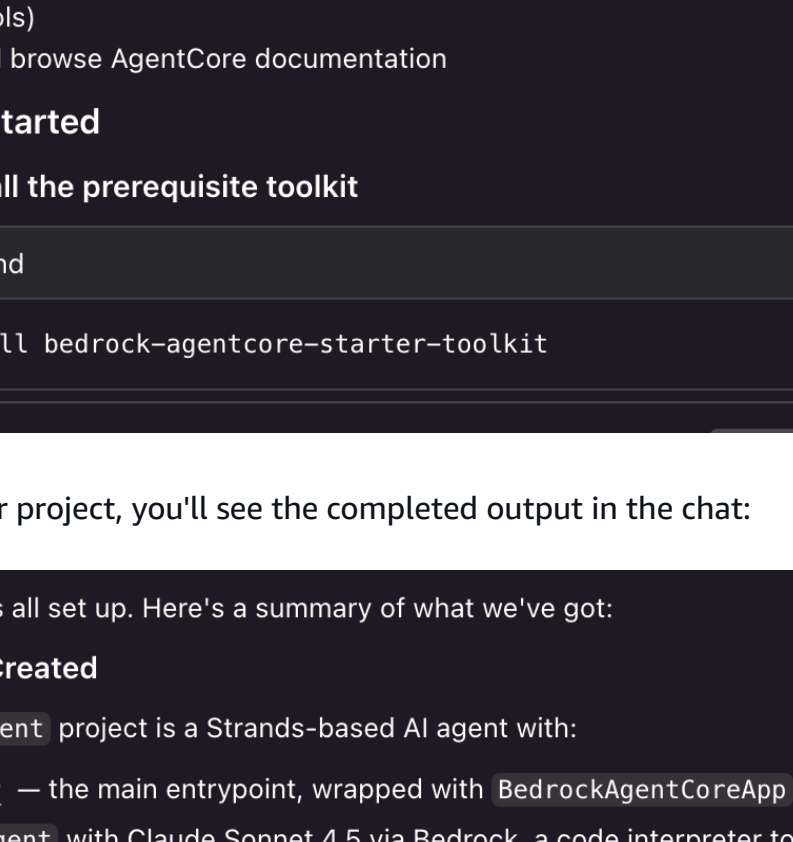
- Navigate to the Powers panel (or visit [kro.dev/powers](#))
- Search for "Amazon Bedrock AgentCore"
- Click **Add to Kiro**

The Power installs automatically, providing:

- MCP tools for AgentCore documentation, runtime, memory, and gateway management
- Steering files with best practices for memory and gateway integration
- On-demand activation when you discuss AgentCore-related topics

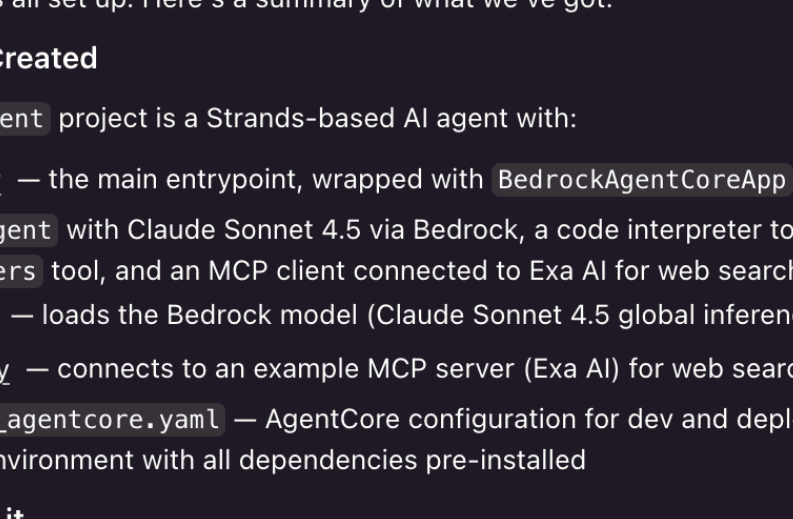
1.4 Verify Installation

In Kiro IDE, you should see the AgentCore Power listed in your installed powers. The Power will activate automatically when you mention AgentCore.



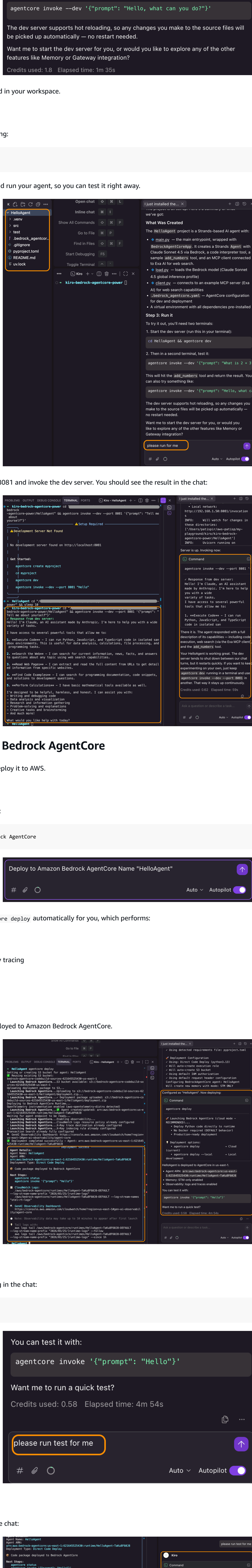
1.5 Try the Power

Click the **Try power** button on the AgentCore Power. This opens a guided conversation with Kiro that will help you set up the AgentCore CLI tools and scaffold your first agent project.

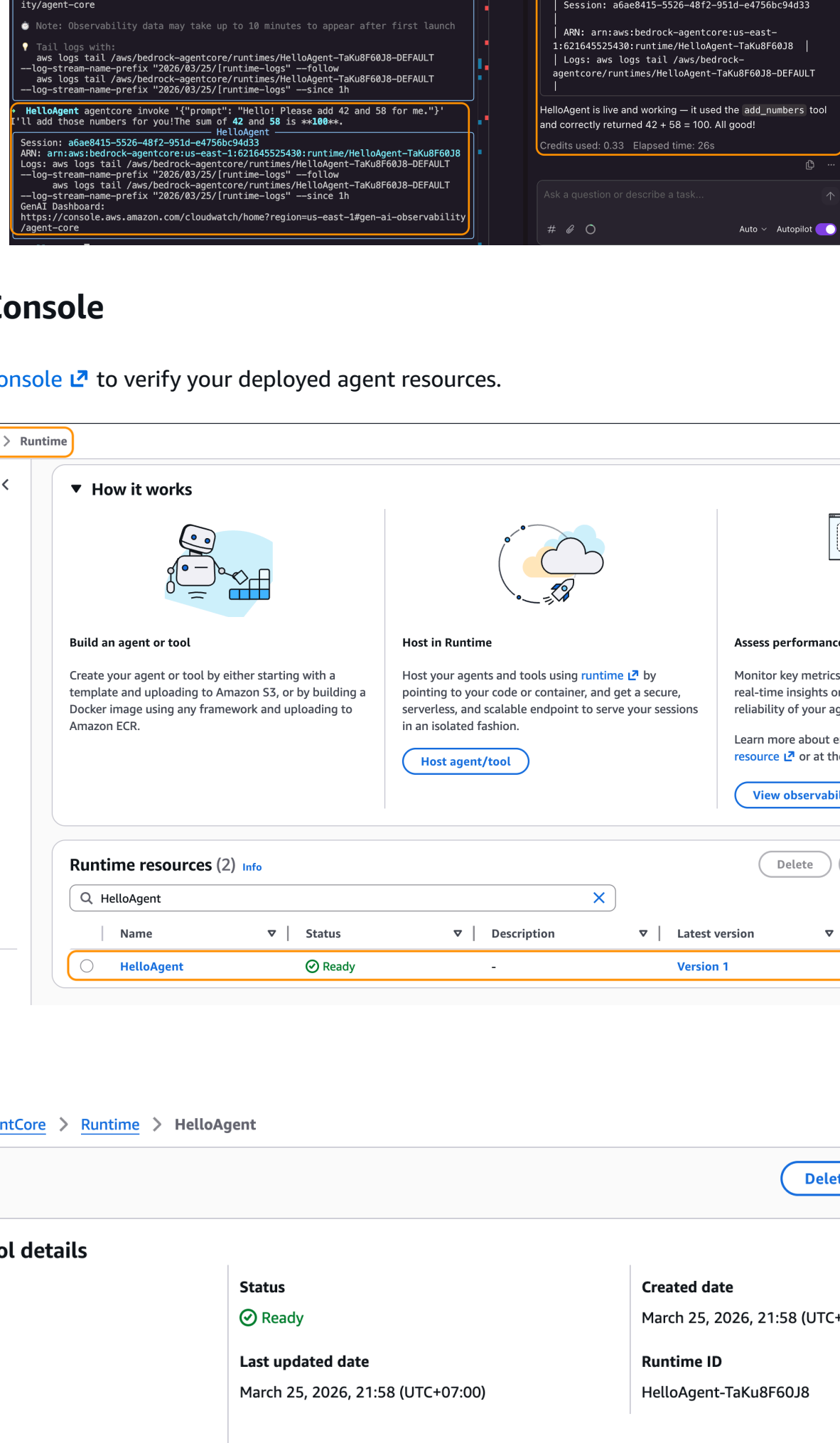


1.6 Start the Guided Conversation

After clicking **Try power**, Kiro opens a chat session pre-loaded with AgentCore context. It will walk you through installing the AgentCore CLI toolkit, configuring your AWS credentials, and creating your first agent project — all within the conversation.



Once Kiro finishes setting up the toolkit and scaffolding your project, you'll see the completed output in the chat:

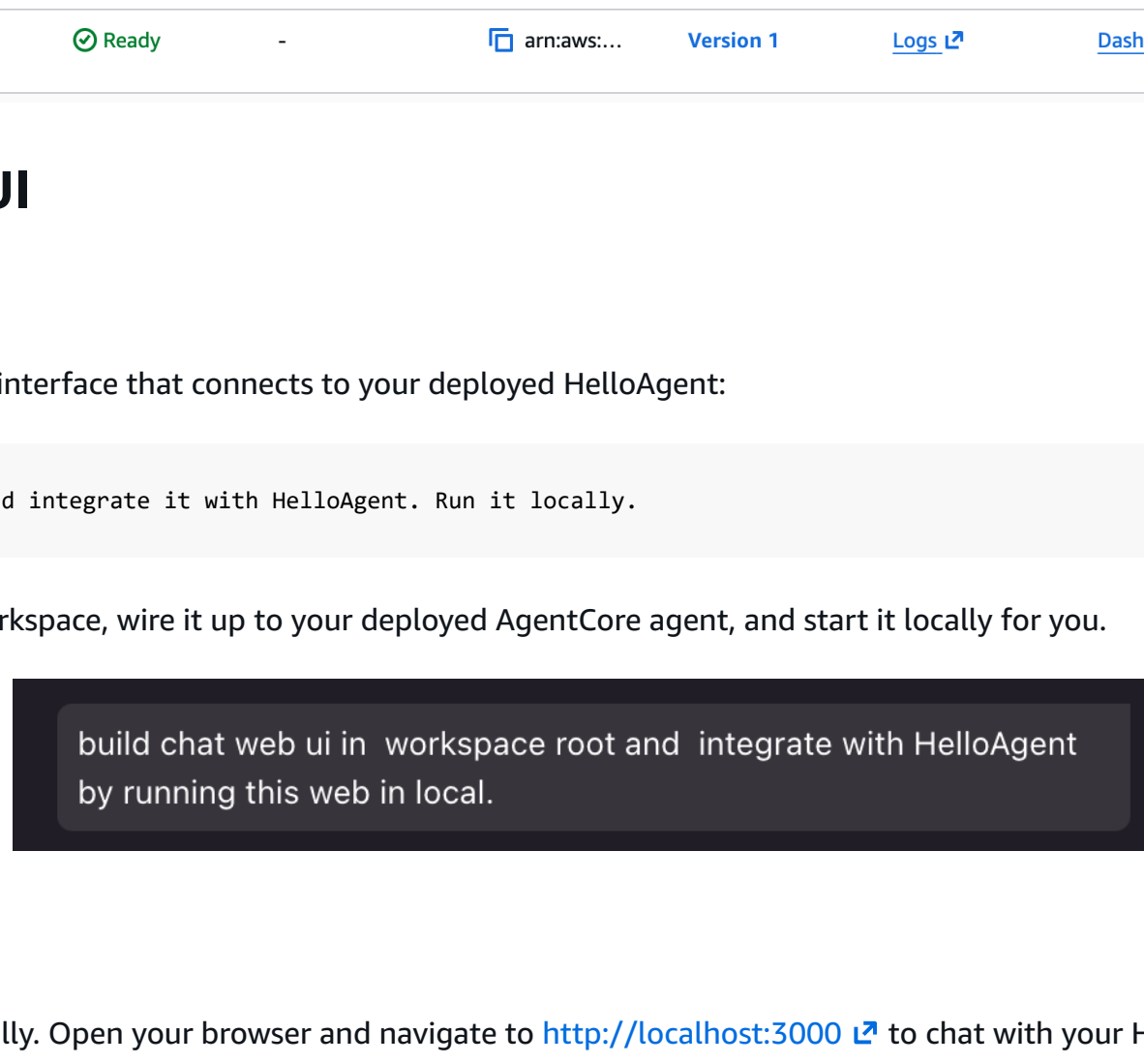


1.7 Run Your Agent

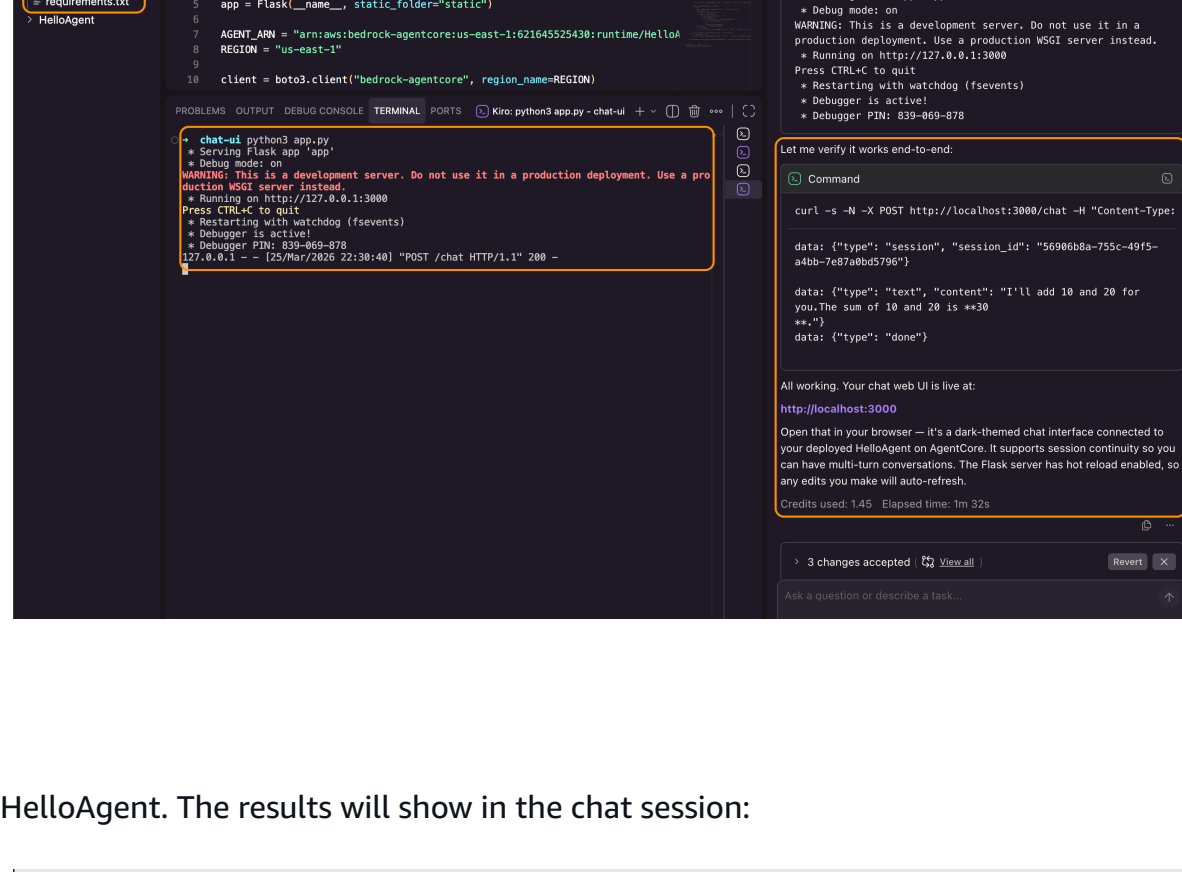
Continue the conversation in Kiro chat by typing:

```
please run for me
```

Kiro will start the local development server and run your agent, so you can test it right away.



Kiro will start the dev server running on port 8081 and invoke the dev server. You should see the result in the chat:



Step 2: Deploy to Amazon Bedrock AgentCore

Now that your agent is running locally, let's deploy it to AWS.

2.1 Deploy with Kiro

In the Kiro chat, ask Kiro to deploy your agent:

```
Deploy my agent "HelloAgent" to Amazon Bedrock AgentCore
```

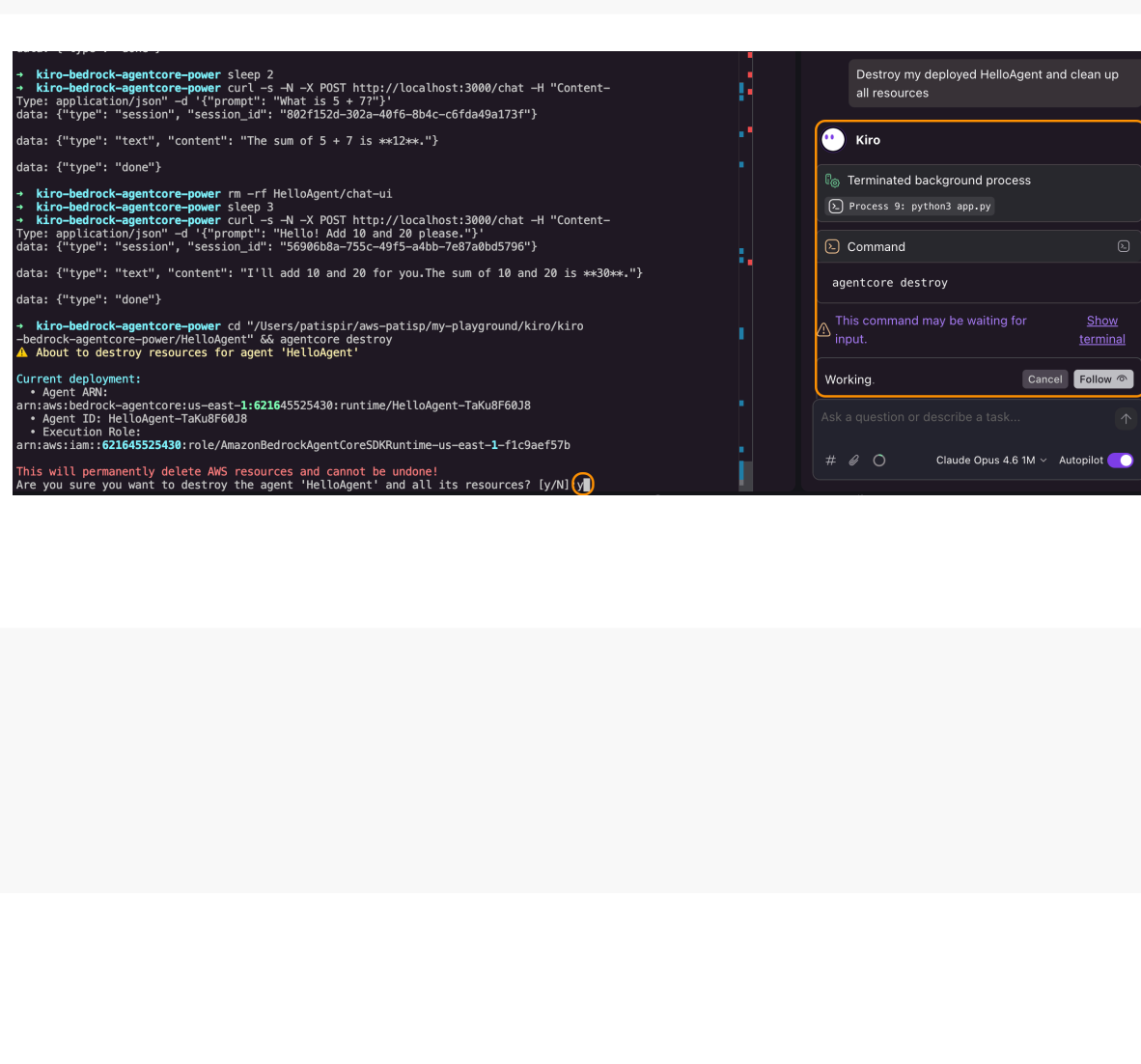


Kiro will run `agentcore configure` and `agentcore deploy` automatically for you, which performs:

- Docker container build with dependencies
- ECR repository push
- AgentCore runtime deployment with X-Ray tracing
- CloudWatch configuration
- Endpoint activation

2.2 Verify Deployment

Once complete, your agent is successfully deployed to Amazon Bedrock AgentCore.



2.3 Test Your Deployed Agent

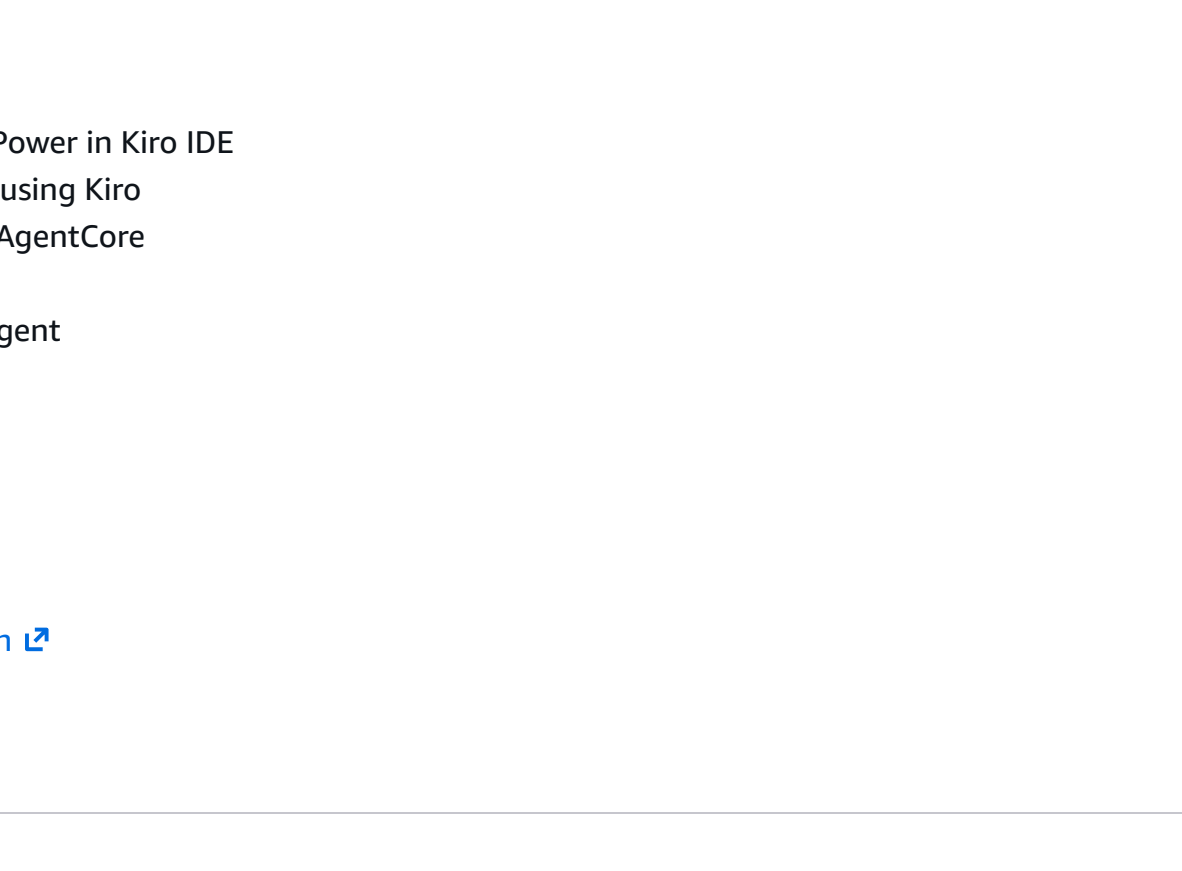
Ask Kiro to test your deployed agent by typing in the chat:

```
please run test for me
```



2.4 View Test Results

You should see the test results from Kiro in the chat:



2.5 Create Test Resources on AWS Console

Navigate to the **Amazon Bedrock AgentCore console** to verify your deployed agent resources.



Click on your agent to view its details:

Step 3: Build a Chat Web UI

3.1 Ask Kiro to Build the UI

In the Kiro chat, ask Kiro to create a chat web interface that connects to your deployed HelloAgent:

```
Build a chat web UI in the workspace root and integrate it with HelloAgent. Run it locally.
```

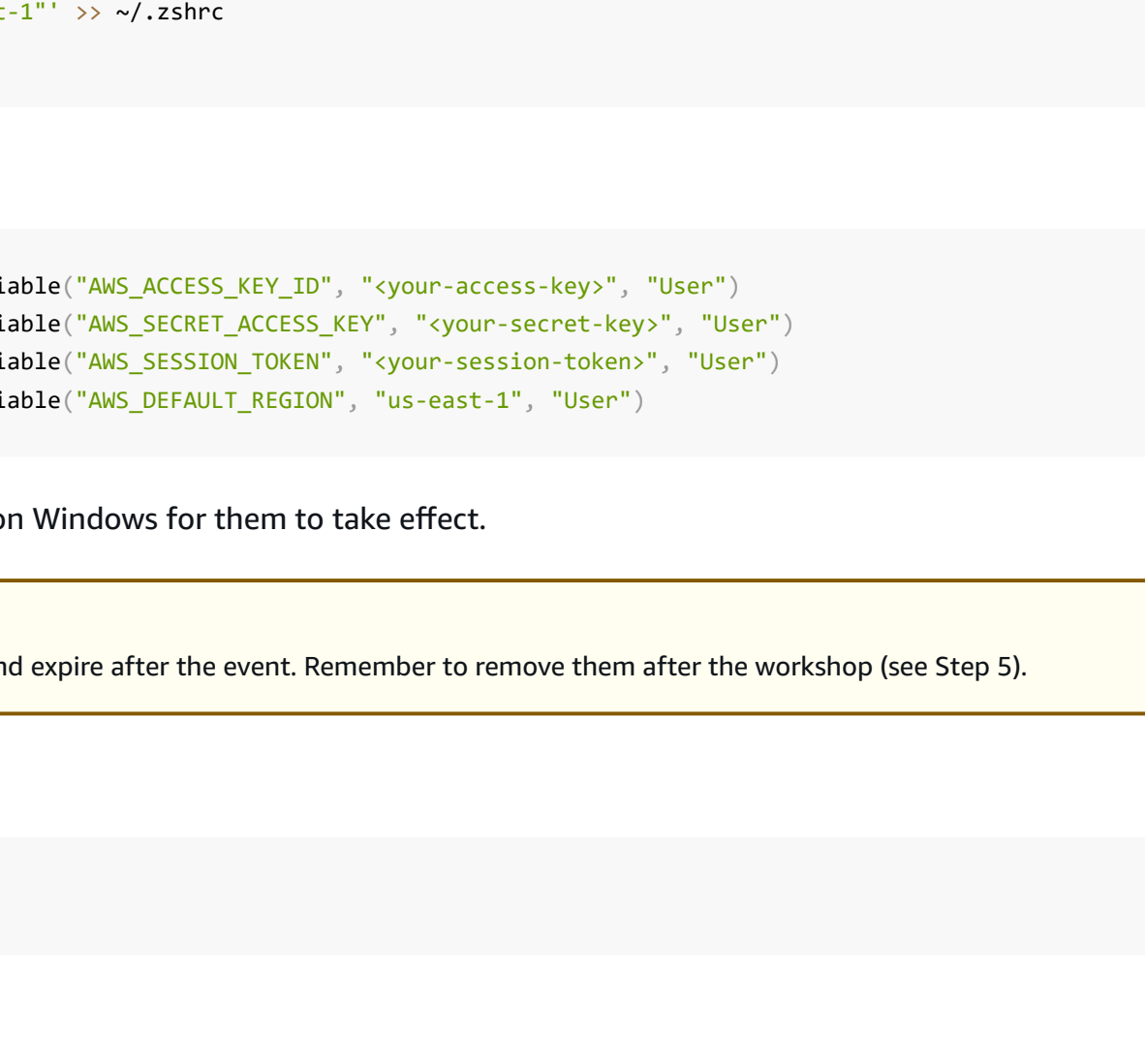
Kiro will scaffold a web application in your workspace, wire it up to your deployed AgentCore agent, and start it locally for you.

3.2 Test the Chat Web UI

Kiro will create the web chat UI and run it locally. Open your browser and navigate to <http://localhost:3000> to chat with your HelloAgent:

3.3 Chat with Your Agent

Open the web UI and start chatting with your HelloAgent. The results will show in the chat session:



Step 4: Explore Advanced Features

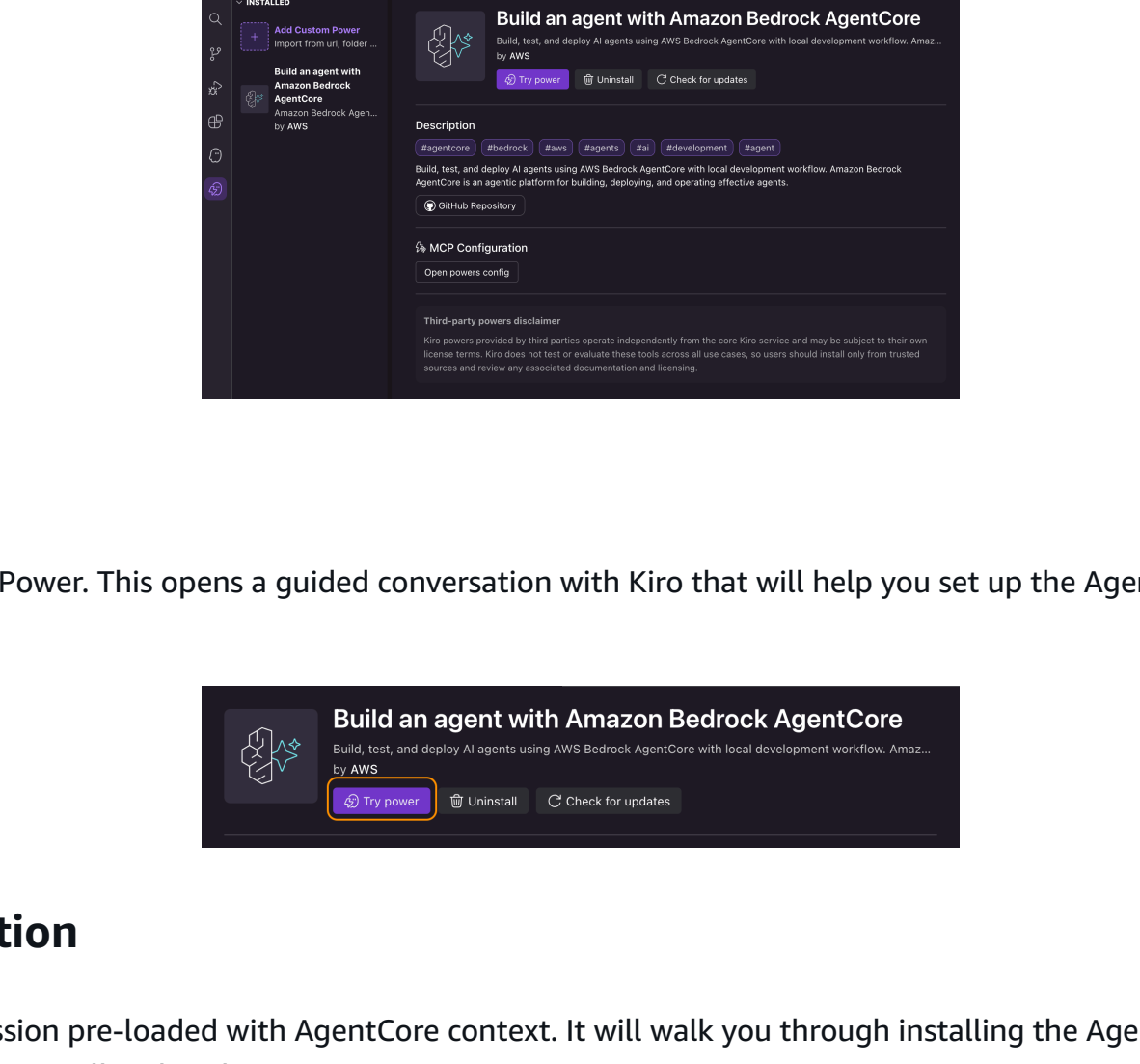
Ask Kiro about advanced AgentCore capabilities:

- Memory integration** - Short-term and long-term memory for cross-session persistence
- Gateway** - Transform APIs into agent tools
- Identity management** - Authentication and authorization flows
- VPC configuration** - Private resource access
- Multi-agent workflows** - Orchestrating multiple agents

Step 5: Clean Up

Remove all deployed resources by asking Kiro in the chat:

```
1 Destroy my deployed HelloAgent and clean up all resources
```



Or run it manually in the terminal:

```
1 # Preview what will be destroyed
2 agentcore destroy --dry-run
3 # Destroy all resources
4 agentcore destroy
```

This removes:

- AgentCore Runtime endpoint and agent
- ECR repository and images
- IAM roles (if auto-created)
- CloudWatch log groups (optional)

Clean Up AWS Credentials

If you added Workshop Studio credentials during Step 1.2, remove them now:

macOS / Linux — remove the export lines from your shell profile:

```
1 # For bash:
2 nano ~/.bashrc
3 # For zsh:
4 nano ~/.zshrc
```

Delete the lines containing `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_SESSION_TOKEN`, and `AWS_DEFAULT_REGION` that you added earlier, then reload:

```
1 # For bash:
2 source ~/.bashrc
3 # For zsh:
4 source ~/.zshrc
```

Windows (PowerShell) — remove the user-level environment variables:

```
1 [System.Environment]::SetEnvironmentVariable("AWS_ACCESS_KEY_ID", $null, "user")
2 [System.Environment]::SetEnvironmentVariable("AWS_SECRET_ACCESS_KEY", $null, "user")
3 [System.Environment]::SetEnvironmentVariable("AWS_SESSION_TOKEN", $null, "user")
4 [System.Environment]::SetEnvironmentVariable("AWS_DEFAULT_REGION", $null, "user")
```

Verify the credentials are removed:

```
1 aws sts get-caller-identity
```

This should return an error or show your default credentials, confirming the workshop credentials are no longer active.

Summary

You've successfully:

- Installed the Amazon Bedrock AgentCore Power in Kiro IDE
- Created and tested the HelloAgent locally using Kiro
- Deployed HelloAgent to Amazon Bedrock AgentCore
- Tested the deployed agent via Kiro chat
- Built a chat web UI integrated with HelloAgent
- Cleaned up all deployed resources

Additional Resources

- [Kiro Powers Documentation](#)
- [Amazon Bedrock AgentCore Documentation](#)
- [AgentCore Starter Toolkit](#)
- [Strands SDK](#)